

Отчёт по лабораторной работе №13

Программирование в командном процессоре ОС UNIX. Ветвления и циклы

Студент: ТУЙИШИМЕ Тьерри

Группа: НКАбд-05-25

Содержание

1 Цель работы 1

2 Задание 1

3 Выполнение лабораторной работы 2

3.1 командный файл, который анализирует командную строку 2

3.2 программа, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю. 3

3.3 командный файл, создающий указанное число файлов 4

3.4 командный файл, который с помощью команды tar запаковывает в архив все файлы в указанной директории. 5

4 Выводы 6

5 Ответы на контрольные вопросы 6

Список литературы 8

1 Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научится писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

2 Задание

1. Используя команды `getopts` `grep`, написать командный файл, который анализирует командную строку
2. Написать на языке Си программу, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю.
3. Написать командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до N
4. Написать командный файл, который с помощью команды `tar` запаковывает в архив все файлы в указанной директории.

3 Выполнение лабораторной работы

3.1 командный файл, который анализирует командную строку

Создаю файл `file1` и в нем написала код, который анализирует командную строку с ключами `-i` (прочитать данные из указанного файла), `-o` (вывести данные в указанный файл), `-p` (указать шаблон для поиска), `-C` (различать большие и малые буквы), `-n` (выдавать номера строк) используя команды `getopts` `grep`:



The screenshot shows a terminal window with a title bar containing "Open", a dropdown arrow, a plus icon, the filename "*file1", the path "~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13", a "Save" button, a menu icon, and a close button. The terminal content is a shell script:

```
1 while getopts "i:o:p:C:n" opt
2 do
3 case $opt in
4 i) inputfile="$OPTARG";;
5 o) outputfile="$OPTARG";;
6 p) template="$OPTARG";;
7 C) register="$OPTARG";;
8 n) number="";;
9 esac
10 done
11
12 grep -n "$template" "$inputfile".txt > "$outputfile".txt
```

Figure 1: Рис. 1: Код для анализа командной строки

```
while getopts "i:o:p:C:n" opt
do
case $opt in
i)inputfile="$OPTARG";;
o)outputfile="$OPTARG";;
p)template="$OPTARG";;
c)register="$OPTARG";;
n)number="";;
esac
done
```

```
grep -n "$template" "$inputfile.txt" > "$outputfile.txt"
```

Далее я установила права на исполнение и запустила файл:

```
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$ gedit file1
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$ cp file1 file1.txt
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$ chmod
chmod: missing operand
Try 'chmod --help' for more information.
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$ chmod
chmod: missing operand
Try 'chmod --help' for more information.
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$ cp file1 file1.txt
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$ chmod +x file1.txt
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$ ls -la file1.txt
-rwxr-xr-x. 1 Micaca Micaca 228 May  7 16:17 file1.txt
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$ gedit file1
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$ cp file1 file1.txt
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$ chmod +x file1.txt
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$
```

Figure 2: Рис. 2: право на исполнение

```
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$ ./file1.txt -i file1 -o output -p n -C -n
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$ cat output.txt
1:while getopts "i:o:p:C:n" opt
3:case $opt in
4:i) inputfile="$OPTARG";;
8:n) number="";;
10:done
12:grep -n "$template" "$inputfile".txt > "$outputfile".txt
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$
```

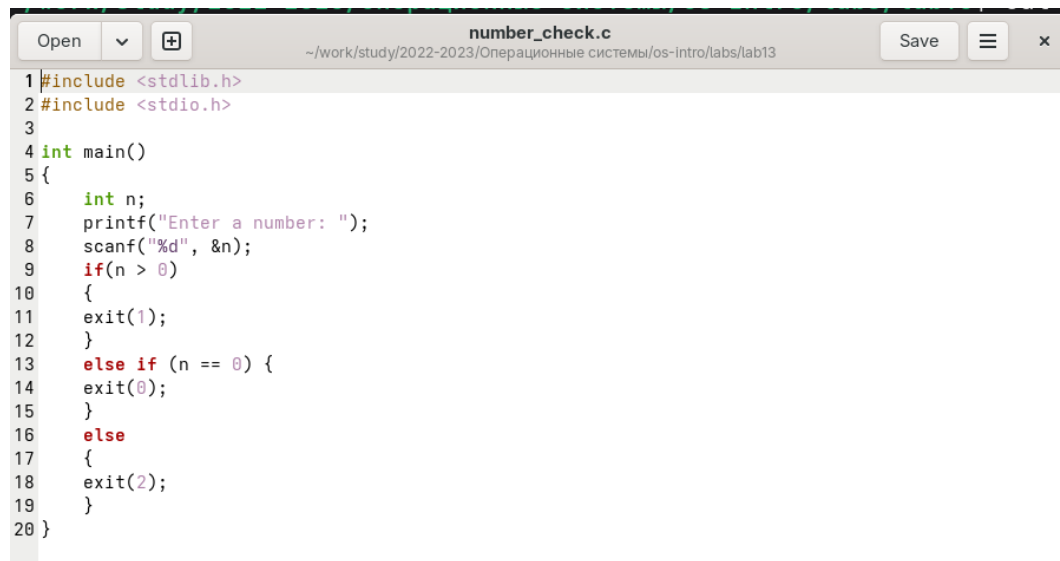
Рис. 3: Запуск file1

3.2 программа, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю.

Написала на языке Си программу, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю. Затем программа завершается с помощью функции `exit(n)`, передавая информацию о коде завершения в оболочку:

```
#include <stdlib.h>
#include <stdio.h>
```

```
int main()
```



```
1 #include <stdlib.h>
2 #include <stdio.h>
3
4 int main()
5 {
6     int n;
7     printf("Enter a number: ");
8     scanf("%d", &n);
9     if(n > 0)
10    {
11        exit(1);
12    }
13    else if (n == 0) {
14        exit(0);
15    }
16    else
17    {
18        exit(2);
19    }
20 }
```

Figure 3: Рис. 4: Программа на языке Си

```
{
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);
    if(n>0)
    {
        exit(1);
    }

    else if (n==0) {
        exit(0);}
    else
    {
        exit(2);
    }
}
```

Далее создала командный файл который вызывает эту программу и, проанализировав с помощью команды \$?, выдает сообщение о том, какое число было введено:

```
gcc -o cprog file2.c
./cprog
```

```
case $? in
0) echo "      ";;
```



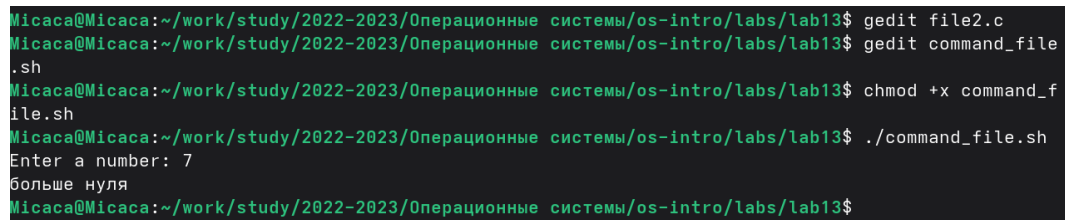
```
1#!/bin/bash
2gcc -o cprog file2.c
3./cprog
4
5case $? in
60) echo "равно нулю";;
71) echo "больше нуля";;
82) echo "меньше нуля";;
9esac
```

Figure 4: Рис. 5: Командный файл программы на Си

```
1) echo "      ";;
2) echo "      ";;
```

```
esac
```

Создала исполняемый файл и запустила:



```
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$ gedit file2.c
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$ gedit command_file
.sh
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$ chmod +x command_f
ile.sh
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$ ./command_file.sh
Enter a number: 7
больше нуля
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$
```

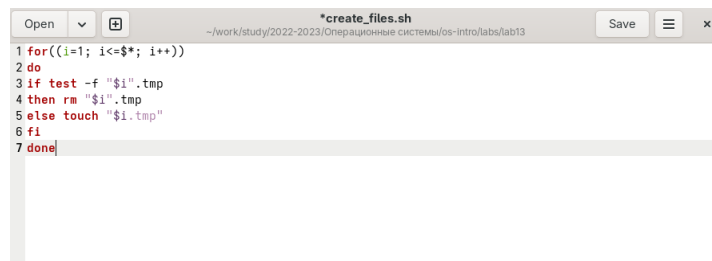
Figure 5: Рис. 6: Результаты программы

3.3 командный файл, создающий указанное число файлов

Я написала командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до . Число файлов, которые необходимо создать, передаётся в аргументы командной строки. Этот же командный файл должен уметь удалять все созданные им файлы (если они существуют):

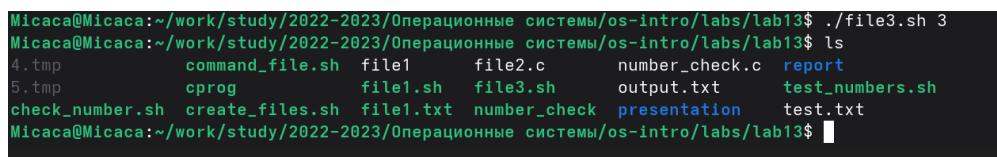
```
for((i=1; i<=$*; i++))
do
if test -f "$i".tmp
then rm "$i".tmp
else touch "$i.tmp"
fi
done
```

Создала исполняемый файл и запустила:



```
1 for((i=1; i<=3; i++))
2 do
3 if test -f "$i".tmp
4 then rm "$i".tmp
5 else touch "$i".tmp"
6 fi
7 done
```

Figure 6: Рис. 7: Командный файл для создания файлов

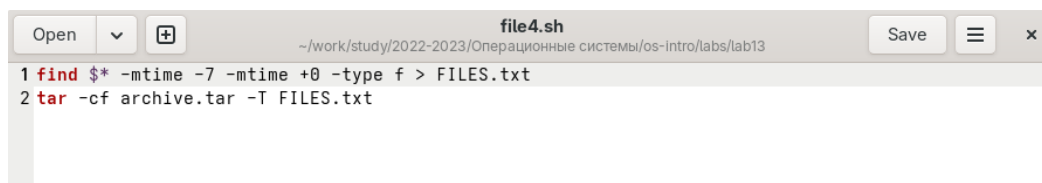


```
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$ ./file3.sh 3
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$ ls
4.tmp          command_file.sh  file1          file2.c        number_check.c  report
5.tmp          cprog           file1.sh       file3.sh       output.txt      test_numbers.sh
check_number.sh create_files.sh  file1.txt      number_check   presentation     test.txt
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$
```

Figure 7: Рис. 8: Создание файлов с помощью командного файла

3.4 командный файл, который с помощью команды tar запаковывает в архив все файлы в указанной директории.

создала командный файл, который с помощью команды tar запаковывает в архив все файлы в указанной директории. Модифицировала его так, чтобы запаковывались только те файлы, которые были изменены менее недели тому назад (использовать команду find).



```
1 find $* -mtime -7 -mtime +0 -type f > FILES.txt
2 tar -cf archive.tar -T FILES.txt
```

Figure 8: Рис. 9: Создание архива

```
find $* -mtime -7 -mtime +0 -type f > FILES.txt
tar -cf archive.tar -T FILES.txt
```

```

Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$ gedit file4.sh
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$ chmod +x file4.sh
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$ ./file4.sh /home/M
icaca/work
tar: Removing leading `/' from member names
tar: Removing leading `/' from hard link targets
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$ ls ~/work
os  study
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$ ls
10.tmp  8.tmp      command_file.sh  file1.sh  file4.sh  output.txt  test.txt
3.tmp   9.tmp      cprog            file1.txt  FILES.txt  presentation
6.tmp   archive.tar create_files.sh  file2.c   number_check  report
7.tmp   check_number.sh file1             file3.sh  number_check.c test_numbers.sh
Micaca@Micaca:~/work/study/2022-2023/Операционные системы/os-intro/labs/lab13$

```

Рис. 10: Результаты кода

4 Выводы

При выполнении проделанной работы я научилась писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

5 Ответы на контрольные вопросы

1. Осуществляет синтаксический анализ командной строки, выделяя флаги, и используется для объявления переменных. Синтаксис команды следующий: `getopts option-string variable`. Флаги – это опции командной строки, обычно помеченные знаком минус; Например, `-F` является флагом для команды `ls -F`. Иногда эти флаги имеют аргументы, связанные с ними. Программы интерпретируют эти флаги, соответствующим образом изменяя свое поведение. Строка опций `option-string` — это список возможных букв и чисел соответствующего флага. Если ожидается, что некоторый флаг будет сопровождаться некоторым аргументом, то за этой буквой должно следовать двоеточие. Соответствующей переменной присваивается буква данной опции. Если команда `getopts` может распознать аргумент, она возвращает истину. Принято включать `getopts` в цикл `while` и анализировать введенные данные с помощью оператора `case`. Предположим, необходимо распознать командную строку следующего формата: `testprog -ifile_in.txt -ofile_out.doc -L -t -r` Вот как выглядит использование оператора `getopts` в этом случае: `while getopts o:i:Ltr optletter do case $optletter in o) iflag = 1; oval =OPTARG;; i) iflag=1; oval=$OPTARG;; L) Lflag=1;; t) tflag=1;; r) rflag=1;; *) echo Illegal option $optletter esac done` Функция `getopts` включает две специальные переменные среды – `OPTARG` и `OPTIND`. Если ожидается дополнительное значение, то `OPTARG` устанавливается в значение этого аргумента (будет равна `file_in.txt` для опции `i` и `file_out.doc` для опции `o`). `OPTIND` является

числовым индексом на упомянутый аргумент. Функция `getopts` также понимает переменные типа массив, следовательно, можно использовать ее в функции не только для синтаксического анализа аргументов функций, но и для анализа введенных пользователем данных.

2. При перечислении имён файлов текущего каталога можно использовать следующие символы: `*` – соответствует произвольной, в том числе и пустой строке; `?` – соответствует любому одинарному символу; `[c1-c2]` – соответствует любому символу, лексикографически находящемуся между символами `c1` и `c2`. Например, `echo *` – выведет имена всех файлов текущего каталога, что представляет собой простейший аналог команды `ls`; `ls .c` – выведет все файлы с последними двумя символами, совпадающими с `.c`. `echo prog.?` – выведет все файлы, состоящие из пяти или шести символов, первыми пятью символами которых являются `prog.`. `[a-z]` – соответствует произвольному имени файла в текущем каталоге, начинающемуся с любой строчной буквы латинского алфавита.
3. Часто бывает необходимо обеспечить проведение каких-либо действий циклически и управление дальнейшими действиями в зависимости отрезультатов проверки некоторого условия. Для решения подобных задач язык программирования `bash` предоставляет возможность использовать такие управляющие конструкции, как `for`, `case`, `if` и `while`. С точки зрения командного процессора эти управляющие конструкции являются обычными командами и могут использоваться как при создании командных файлов, так и при работе в интерактивном режиме. Команды, реализующие подобные конструкции, по сути, являются операторами языка программирования `bash`. Поэтому при описании языка программирования `bash` термин оператор будет использоваться наравне с термином команда. Команды ОС UNIX возвращают код завершения, значение которого может быть использовано для принятия решения о дальнейших действиях. Команда `test`, например, создана специально для использования в командных файлах. Единственная функция этой команды заключается в выработке кода завершения.
4. Два несложных способа позволяют вам прерывать циклы в оболочке `bash`. Команда `break` завершает выполнение цикла, а команда `continue` завершает данную итерацию блока операторов. Команда `break` полезна для завершения цикла `while` в ситуациях, когда условие перестаёт быть правильным. Команда `continue` используется в ситуациях, когда больше нет необходимости выполнять блок операторов, но вы можете захотеть продолжить проверять данный блок на других условных выражениях.
5. Следующие две команды ОС UNIX используются только совместно с управляющими конструкциями языка программирования `bash`: это команда `true`, которая всегда возвращает код завершения, равный нулю (т.е. истина), и команда `false`, которая всегда возвращает код завершения, не равный нулю (т. е. ложь).

6. Строка `if test -f mans/i.s, mans/i.s` и является ли этот файл обычным файлом. Если данный файл является каталогом, то команда вернет нулевое значение (ложь).
7. Выполнение оператора цикла `while` сводится к тому, что сначала выполняется последовательность команд (операторов), которую задаёт список-команд в строке, содержащей служебное слово `while`, а затем, если последняя выполненная команда из этой последовательности команд возвращает нулевой код завершения (истина), выполняется последовательность команд (операторов), которую задаёт список-команд в строке, содержащей служебное слово `do`, после чего осуществляется безусловный переход на начало оператора цикла `while`. Выход из цикла будет осуществлён тогда, когда последняя выполненная команда из последовательности команд (операторов), которую задаёт список-команд в строке, содержащей служебное слово `while`, возвратит ненулевой код завершения (ложь). При замене в операторе цикла `while` служебного слова `while` на `until` условие, при выполнении которого осуществляется выход из цикла, меняется на противоположное. В остальном оператор цикла `while` и оператор цикла `until` идентичны.

Список литературы

Архитектура коипьютеров